

GOVERNO DO ESTADO DE SÃO PAULO
CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA
FACULDADE DE TECNOLOGIA ZONA SUL
CURSO SUPERIOR DE TECNOLOGIA EM
DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

DOCUMENTO DE VISÃO E REQUISITOS

Projeto DSM Conecta: aplicativo multiplataforma de divulgação do curso com coleta e análise de dados em tempo real

Disciplina: Laboratório de Desenvolvimento de Software Multiplataforma

Docente responsável: Prof. Dr. Winston Aparecido Andrade

Semestre letivo: 2026/2

Versão 1.0

SÃO PAULO

2026

SUMÁRIO

SUMÁRIO.....	2
1 INTRODUÇÃO	4
1.1 Contexto	4
1.2 Justificativa	4
1.3 Objetivos.....	4
1.3.1 Objetivo geral	4
1.3.2 Objetivos específicos	5
2 VISÃO DO PRODUTO	5
2.1 Descrição geral	5
2.2 Partes interessadas	6
2.3 Escopo.....	6
2.4 Fora de escopo	6
3 REQUISITOS FUNCIONAIS	7
3.1 Módulo de conteúdo público	7
3.2 Módulo de interação e eventos	7
3.3 Módulo de telemetria e captura de dados.....	8
3.4 Módulo analítico e administrativo.....	8
4 REQUISITOS NÃO FUNCIONAIS	9
4.1 Portabilidade e interoperabilidade	9
4.2 Desempenho e escalabilidade	9
4.3 Concorrência e transparência	10
4.4 Segurança, privacidade e conformidade.....	10
4.5 Manutenibilidade e usabilidade.....	10
5 REGRAS DE NEGÓCIO	11
6 ARQUITETURA DA SOLUÇÃO	11

6.1 Visão geral.....	11
6.2 Componentes	12
6.3 Contrato de mensagens.....	12
6.4 Modelo de dados	13
6.5 Tecnologias adotadas.....	13
7 ADERÊNCIA AOS CONCEITOS DE SISTEMAS DISTRIBUÍDOS	14
8 ESTRATÉGIA DE DESENVOLVIMENTO	15
8.1 Processo e organização das equipes	15
8.2 Desenvolvimento dirigido a testes	15
8.3 Controle de versão e integração contínua	16
8.4 Cronograma.....	16
9 RESPONSABILIDADE SOCIAL E ASPECTOS ÉTICOS.....	17
10 RISCOS E MEDIDAS DE CONTENÇÃO	17
11 CRITÉRIOS DE ACEITAÇÃO DO PRODUTO	18
REFERÊNCIAS.....	19

1 INTRODUÇÃO

1.1 Contexto

O curso superior de tecnologia em Desenvolvimento de Software Multiplataforma forma profissionais capazes de projetar e construir aplicações que operam simultaneamente em dispositivos móveis, navegadores e computadores pessoais. Apesar da aderência do curso às demandas do mercado, boa parte dos estudantes do ensino médio desconhece a existência dessa formação, seus conteúdos e as possibilidades profissionais que ela abre.

A disciplina Laboratório de Desenvolvimento de Software Multiplataforma propõe que os próprios estudantes construam a solução para esse problema. O projeto descrito neste documento consiste no desenvolvimento de um aplicativo de divulgação do curso, acompanhado de uma infraestrutura de coleta e análise de dados em tempo real que permite avaliar o alcance das ações de divulgação.

1.2 Justificativa

A escolha do tema atende a três necessidades simultâneas. A primeira é pedagógica, pois o projeto exige a aplicação integrada de engenharia de software, programação, sistemas distribuídos e gerência de projetos em um mesmo produto, o que reproduz as condições de um desenvolvimento profissional. A segunda é institucional, já que a coordenação do curso passa a dispor de um instrumento próprio de divulgação e de indicadores objetivos sobre o interesse do público. A terceira é social, uma vez que a ação de divulgação em escolas públicas de ensino médio amplia o acesso à informação sobre formação em tecnologia entre jovens que raramente recebem essa orientação.

O projeto também dispensa a aquisição de equipamentos. A captura de dados por sensores é realizada pelos recursos já presentes nos aparelhos dos usuários e por nós de sensoriamento simulados, o que elimina custo e risco de atraso na execução.

1.3 Objetivos

1.3.1 Objetivo geral

Desenvolver, ao longo do semestre letivo, uma aplicação multiplataforma de divulgação do curso de Desenvolvimento de Software Multiplataforma, integrada a uma arquitetura distribuída de coleta, armazenamento e análise de dados em tempo real.

1.3.2 Objetivos específicos

- Especificar requisitos e arquitetura antes da codificação, adotando contratos de interface estáveis entre as equipes.
- Implementar o produto para dispositivo móvel, navegador e computador pessoal a partir de uma única base de código.
- Aplicar protocolo de mensageria e broker para o transporte dos dados de telemetria entre os componentes do sistema.
- Persistir grande volume de dados de série temporal com estratégias de particionamento, agregação e retenção.
- Processar o fluxo de dados em tempo real com técnicas de janelamento e detecção de anomalia.
- Conduzir o desenvolvimento sob a prática de desenvolvimento dirigido a testes e controle de versão distribuído.
- Realizar ação de divulgação junto a escolas públicas de ensino médio, com devolutiva dos resultados aos parceiros.

2 VISÃO DO PRODUTO

2.1 Descrição geral

O produto recebe o nome provisório de DSM Conecta. Trata-se de um aplicativo de divulgação institucional voltado a candidatos, estudantes do ensino médio, famílias e professores da rede pública, acompanhado de um painel analítico destinado à coordenação do curso e de um painel público de métricas agregadas.

Do ponto de vista do visitante, o aplicativo apresenta o curso, a matriz curricular, os projetos desenvolvidos pelos estudantes, a agenda de eventos e um questionário de afinidade vocacional. Do ponto de vista da instituição, cada interação relevante

gera um evento de telemetria que alimenta indicadores de alcance e de interesse em tempo real.

2.2 Partes interessadas

- **Público visitante.** Candidatos e estudantes do ensino médio, que consultam informações sobre o curso e recebem orientação vocacional.
- **Coordenação do curso.** Aprova conteúdos, acompanha os indicadores e utiliza os dados no planejamento das ações de divulgação.
- **Estudantes do curso.** Fornecem conteúdo de vitrine e depoimentos, além de atuarem como monitores nas ações presenciais.
- **Escolas parceiras.** Recebem a ação de divulgação e a devolutiva dos dados coletados durante a visita.
- **Equipe de desenvolvimento.** Turma da disciplina, responsável pela concepção, construção, teste e entrega do produto.

2.3 Escopo

Integram o escopo do projeto os seguintes elementos:

- Aplicativo cliente compilado para Android, navegador web e desktop.
- Serviço de back-end com interface de programação documentada e canal de comunicação em tempo real.
- Broker de mensageria e serviço de ingestão concorrente.
- Banco de dados de série temporal com rotinas de agregação e retenção.
- Painel administrativo e painel público de métricas.
- Nó sensor simulado e gerador de carga sintética para os ensaios de escalabilidade.
- Documentação técnica, suíte de testes automatizados e pipeline de integração contínua.

2.4 Fora de escopo

Não integram o escopo desta versão:

- Integração com o sistema acadêmico institucional ou com o sistema de inscrição em processo seletivo.
- Publicação nas lojas oficiais de aplicativos, que depende de contas institucionais.
- Aquisição, montagem ou instalação de equipamentos físicos de sensoriamento.
- Versão para iOS, dependente de infraestrutura de compilação específica.
- Cadastro de usuário com dado pessoal identificável para o público visitante.

3 REQUISITOS FUNCIONAIS

Os requisitos funcionais estão organizados por módulo e identificados pelo prefixo RF, para permitir a rastreabilidade entre backlog, código e testes automatizados.

3.1 Módulo de conteúdo público

- RF01** Exibir a apresentação institucional do curso, contemplando perfil do egresso, duração, turnos e formas de ingresso.
- RF02** Disponibilizar a matriz curricular com a descrição resumida de cada disciplina.
- RF03** Apresentar vitrine de projetos desenvolvidos pelos estudantes, com imagem, descrição e tecnologias empregadas.
- RF04** Exibir depoimentos de estudantes e egressos em formato de texto e imagem.
- RF05** Apresentar a agenda de eventos, prazos e etapas do processo seletivo.
- RF06** Disponibilizar canal de dúvidas com perguntas frequentes e envio de mensagem à coordenação.
- RF07** Permitir o compartilhamento de conteúdos do aplicativo em aplicativos de mensagem e redes sociais.
- RF08** Operar com o conteúdo armazenado localmente quando não houver conexão de rede.

3.2 Módulo de interação e eventos

- RF09** Permitir que o visitante responda ao questionário de afinidade vocacional e receba um resultado orientativo.
- RF10** Registrar a presença do visitante em evento de divulgação por leitura de código QR.
- RF11** Registrar a presença por proximidade geográfica quando o visitante autorizar o uso da localização.
- RF12** Apresentar ao visitante o histórico das próprias interações e permitir a exclusão dos dados associados à sessão.

3.3 Módulo de telemetria e captura de dados

- RF13** Publicar cada interação relevante do aplicativo como mensagem em tópico do broker de mensageria.
- RF14** Coletar leituras dos sensores do dispositivo, tais como localização aproximada, luminosidade e movimento, mediante consentimento prévio.
- RF15** Receber leituras provenientes de nó sensor externo simulado, responsável pela contagem de visitantes no estande do curso.
- RF16** Validar o formato e a versão do esquema de cada mensagem recebida antes da persistência.
- RF17** Descartar mensagens duplicadas ou malformadas, registrando a ocorrência em log estruturado.
- RF18** Persistir as mensagens válidas na base de série temporal preservando o instante de origem.

3.4 Módulo analítico e administrativo

- RF19** Exibir, em tempo real, a contagem de visitantes ativos e de registros de presença por evento.
- RF20** Apresentar o ranking dos conteúdos mais acessados em janela deslizante configurável.
- RF21** Exibir a distribuição dos resultados do questionário de afinidade vocacional.
- RF22** Emitir alerta quando o volume de acessos ultrapassar o limiar estatístico definido para a janela corrente.

RF23 Permitir a gestão do conteúdo do aplicativo, com inclusão, alteração e remoção por usuário autenticado.

RF24 Permitir a exportação dos dados agregados nos formatos CSV e JSON.

RF25 Publicar painel público de métricas agregadas e anonimizadas, acessível sem autenticação.

4 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais expressam as restrições de qualidade do sistema e orientam boa parte das decisões de arquitetura descritas na seção 6.

4.1 Portabilidade e interoperabilidade

RNF01 O aplicativo cliente deve ser gerado para Android, navegador web e desktop a partir de uma única base de código.

RNF02 As interfaces de programação devem ser descritas em OpenAPI 3, e o transporte de telemetria deve seguir o protocolo MQTT na versão 3.1.1 ou superior.

RNF03 Os dados abertos devem ser publicados em formatos não proprietários, com dicionário de dados disponível.

4.2 Desempenho e escalabilidade

RNF04 Noventa e cinco por cento das requisições à interface de programação devem ser respondidas em até 500 milissegundos sob carga nominal.

RNF05 Um evento publicado no broker deve estar visível no painel em tempo real em até 3 segundos.

RNF06 O sistema deve sustentar 1000 clientes conectados e 50 mensagens por segundo sem perda de mensagens em qualidade de serviço 1.

RNF07 O serviço de ingestão deve permitir a execução de múltiplas instâncias simultâneas sem duplicação de registros.

RNF08 A base de dados deve manter o tempo de consulta dos painéis abaixo de 2 segundos com pelo menos um milhão de registros armazenados.

4.3 Concorrência e transparência

RNF09 A ingestão de dados deve ser assíncrona e concorrente, com tratamento de contrapressão quando a taxa de chegada superar a de gravação.

RNF10 O cliente deve permanecer alheio à localização física dos serviços consumidos, atendendo às transparências de acesso e de localização.

RNF11 A indisponibilidade temporária do broker não deve interromper o uso do aplicativo, que armazena os eventos localmente para envio posterior.

4.4 Segurança, privacidade e conformidade

RNF12 O acesso administrativo deve exigir autenticação por token e trafegar sobre canal cifrado.

RNF13 Credenciais e segredos devem residir em variáveis de ambiente, nunca no repositório de código.

RNF14 A coleta de dados deve ser precedida de consentimento explícito, informado e revogável, em conformidade com a Lei nº 13.709/2018.

RNF15 A identificação do visitante deve ocorrer por identificador aleatório de sessão, sem vinculação a dado pessoal.

RNF16 O painel público não deve exibir informação que permita a reidentificação de um visitante.

4.5 Manutenibilidade e usabilidade

RNF17 A cobertura de testes automatizados do back-end deve ser de no mínimo setenta por cento das linhas.

RNF18 A integração contínua deve impedir a incorporação de alterações com teste falhando.

RNF19 A interface deve atender a requisitos básicos de acessibilidade, com contraste adequado, rótulos para leitores de tela e suporte ao aumento do tamanho de fonte.

RNF20 Os dados brutos de telemetria devem ser retidos por noventa dias e os dados agregados por cinco anos.

5 REGRAS DE NEGÓCIO

- RN01** Nenhum dado de telemetria é coletado antes do aceite do termo de consentimento pelo visitante.
- RN02** O identificador de sessão é gerado localmente, de forma aleatória, e é descartado quando o visitante solicita a exclusão dos dados.
- RN03** O registro de presença é válido apenas dentro da janela de horário definida para o evento.
- RN04** O registro de presença por geolocalização exige que o dispositivo esteja a até cento e cinquenta metros do local do evento.
- RN05** Cada sessão registra no máximo uma presença por evento.
- RN06** O resultado do questionário de afinidade é orientativo e não produz qualquer efeito sobre processo seletivo.
- RN07** A publicação de conteúdo no aplicativo depende de aprovação prévia da coordenação do curso.
- RN08** Os dados abertos são divulgados somente em nível agregado, com no mínimo dez registros por grupo apresentado.

6 ARQUITETURA DA SOLUÇÃO

6.1 Visão geral

A solução adota arquitetura distribuída orientada a eventos, organizada em quatro camadas. A camada de borda é formada pelos aplicativos clientes e pelo nó sensor simulado, responsáveis pela produção dos dados. A camada de transporte é composta pelo broker de mensageria. A camada de processamento reúne o serviço de ingestão, o motor de análise em tempo real e a interface de programação. A camada de persistência abriga o banco de série temporal e o repositório de conteúdo.

O desacoplamento entre produção e consumo de dados é intencional. Ele permite que as equipes trabalhem em paralelo, que novos produtores sejam acrescentados sem alteração no restante do sistema e que o serviço de ingestão seja replicado quando a carga aumentar.

6.2 Componentes

- Aplicativo cliente, desenvolvido em base única e compilado para as três plataformas exigidas.
- Broker de mensageria, responsável pelo roteamento das mensagens entre produtores e consumidores.
- Serviço de ingestão, que assina os tópicos, valida, deduplica e persiste as mensagens.
- Banco de dados de série temporal, com tabelas particionadas por tempo e agregações contínuas.
- Interface de programação, que expõe conteúdo, histórico e métricas por requisições e por canal persistente.
- Motor de análise em tempo real, que calcula janelas deslizantes, médias móveis e detecção de anomalia.
- Painel administrativo, destinado à coordenação, e painel público de métricas agregadas.
- Nó sensor simulado e gerador de carga sintética, empregados nos ensaios de escalabilidade.

6.3 Contrato de mensagens

A estrutura hierárquica dos tópicos segue o padrão `dsm/ambiente/origem/categoria/identificador`, o que permite assinaturas seletivas por curinga. São exemplos de tópicos previstos:

- `dsm/prod/app/interacao/tela`, para a abertura de telas do aplicativo.
- `dsm/prod/app/quiz/resposta`, para as respostas do questionário de afinidade.
- `dsm/prod/app/presenca/evento`, para os registros de presença em eventos.
- `dsm/prod/app/sensor/dispositivo`, para as leituras dos sensores do aparelho.
- `dsm/prod/totem/sensor/contagem`, para as leituras do nó sensor simulado.

Cada mensagem trafega em notação JSON e contém, no mínimo, o identificador do evento, o identificador de sessão, a categoria, a origem, o instante de geração, a versão do esquema e a carga útil específica. A versão do esquema é obrigatória porque permite a evolução do contrato sem interromper produtores antigos, o que materializa a propriedade de openness prevista na ementa da disciplina.

6.4 Modelo de dados

O modelo combina dados relacionais convencionais e dados de série temporal. As entidades de conteúdo e configuração seguem o modelo relacional tradicional, enquanto os eventos de telemetria são armazenados em tabela particionada por tempo:

- Entidades relacionais: conteúdo, projeto, depoimento, evento, questão, alternativa e usuário administrativo.
- Entidade de série temporal: telemetria, com particionamento por intervalo de tempo e índice composto por categoria e instante.
- Visões materializadas: agregados por hora e por dia, atualizados de forma incremental.
- Políticas de ciclo de vida: compressão dos dados brutos após trinta dias e remoção após noventa dias.

6.5 Tecnologias adotadas

- Aplicativo cliente em Flutter, com geração para Android, web e desktop a partir da mesma base.
- Broker Eclipse Mosquitto, executado em contêiner.
- Serviço de ingestão em Python, com biblioteca cliente MQTT e execução assíncrona.
- Interface de programação em FastAPI, com documentação automática em OpenAPI e canal em tempo real por WebSocket.
- Banco PostgreSQL com extensão de série temporal para particionamento e agregação contínua.

- Orquestração local por Docker Compose, de modo que todo o ambiente seja iniciado por um único comando.
- Testes automatizados com pytest no back-end e com a suíte nativa do Flutter no cliente.
- Controle de versão em Git, repositório remoto no GitHub e integração contínua por GitHub Actions.
- Nó sensor simulado em ambiente de simulação de microcontrolador acessível pelo navegador, sem necessidade de placa física.

7 ADERÊNCIA AOS CONCEITOS DE SISTEMAS DISTRIBUÍDOS

A ementa da disciplina exige o tratamento explícito de propriedades de sistemas distribuídos. O quadro conceitual abaixo relaciona cada propriedade ao elemento correspondente do projeto:

- **Concorrência:** o serviço de ingestão processa múltiplos tópicos simultaneamente e a interface de programação atende requisições concorrentes, com controle de acesso a recursos compartilhados e tratamento de contrapressão.
- **Openness:** o uso de protocolos e formatos abertos, somado à documentação pública das interfaces e à versão explícita do esquema de mensagens, permite que novos produtores e consumidores se integrem sem alteração no núcleo do sistema.
- **Escalabilidade:** o particionamento temporal dos dados, as agregações contínuas e a possibilidade de replicar o serviço de ingestão permitem o crescimento da carga sem redesenho da solução, o que é verificado por ensaio com carga sintética.
- **Transparência:** o cliente desconhece a localização e a tecnologia dos serviços que consome, e a falha temporária de um componente é absorvida por armazenamento local e retentativa.
- **Mensageria:** a comunicação assíncrona por publicação e assinatura desacopla produtores e consumidores no tempo e no espaço, o que permite que equipes distintas evoluam seus módulos de forma independente.

- **Mineração em tempo real:** o processamento em janelas deslizantes, o cálculo de médias móveis e a detecção de desvios estatísticos operam sobre o fluxo, sem depender de consulta ao histórico completo.

8 ESTRATÉGIA DE DESENVOLVIMENTO

8.1 Processo e organização das equipes

O projeto é conduzido em iterações de duas a três semanas, com reunião de planejamento no início, acompanhamento semanal e revisão ao final de cada iteração. A turma é dividida em equipes por camada arquitetural, com um papel rotativo de responsável pela integração e pelo repositório:

- Equipe de captura de dados, responsável pelo nó sensor simulado e pela leitura dos sensores do dispositivo.
- Equipe de infraestrutura e ingestão, responsável pelo broker, pelo serviço de ingestão e pela orquestração dos contêineres.
- Equipe de dados e análise, responsável pela modelagem, pelas agregações e pelo motor de análise em tempo real.
- Equipe de aplicações, responsável pelo aplicativo cliente, pelo painel administrativo e pelo painel público.

Os contratos definidos na primeira iteração, isto é, o esquema de mensagens e a especificação da interface de programação, tornam-se estáveis a partir de sua aprovação. Qualquer alteração posterior exige solicitação formal de incorporação revisada por integrante de outra equipe, o que evita bloqueios recíprocos.

8.2 Desenvolvimento dirigido a testes

O desenvolvimento dirigido a testes é adotado desde a segunda iteração e incide obrigatoriamente sobre os módulos de validação de mensagem, de deduplicação, de cálculo das janelas de análise e das regras de negócio. O ciclo segue as três etapas canônicas: escrita do teste que falha, implementação mínima que o faz passar e refatoração com a suíte verde.

A exigência de teste anterior ao código de produção é verificada pelo histórico do repositório, e não apenas pela existência da suíte ao final da iteração.

8.3 Controle de versão e integração contínua

- Repositório único, com ramo principal protegido e ramos de funcionalidade nomeados por identificador de tarefa.
- Incorporação de alterações somente por solicitação revisada, com pelo menos uma aprovação de integrante de outra equipe.
- Mensagens de commit padronizadas, o que permite a geração automática do registro de alterações.
- Pipeline de integração contínua executando análise estática e a suíte de testes a cada envio.
- Marcação de versão ao final de cada iteração, com registro das entregas correspondentes.

8.4 Cronograma

O projeto é executado em sete iterações distribuídas ao longo de dezesseis semanas letivas, entre agosto e novembro de 2026:

- **Iteração 0.** Semanas 1 a 3. Iniciação, levantamento de requisitos, arquitetura, contratos e ambiente de desenvolvimento.
- **Iteração 1.** Semanas 4 e 5. Controle de versão, integração contínua e fundação da prática de testes.
- **Iteração 2.** Semanas 6 e 7. Broker de mensageria, produtores de dados e nó sensor simulado.
- **Iteração 3.** Semanas 8 e 9. Serviço de ingestão concorrente e persistência em série temporal.
- **Iteração 4.** Semanas 10 e 11. Interface de programação e motor de análise em tempo real.
- **Iteração 5.** Semanas 12 a 14. Aplicativo cliente nas três plataformas e painel administrativo.

- **Iteração 6.** Semanas 15 e 16. Ensaio de carga, documentação final, ação de divulgação e apresentação pública.

9 RESPONSABILIDADE SOCIAL E ASPECTOS ÉTICOS

O projeto não se encerra na entrega técnica. A última iteração prevê uma ação presencial de divulgação em escola pública parceira ou no próprio campus, na qual os estudantes apresentam o curso, aplicam o questionário de afinidade e discutem trajetórias profissionais em tecnologia com os visitantes. Os dados coletados durante a ação são devolvidos à escola parceira em relatório agregado, o que caracteriza troca efetiva de saberes e não simples coleta.

O tratamento de dados pessoais é conduzido como requisito de projeto. O consentimento é solicitado de forma clara antes de qualquer coleta, o visitante pode consultar e excluir os dados da própria sessão, a identificação ocorre por código aleatório e o painel público exibe apenas agregações que impedem a reidentificação. Essas decisões são discutidas em aula como aplicação prática da Lei Geral de Proteção de Dados Pessoais, e não como formalidade burocrática.

A publicação dos dados agregados em formato aberto, acompanhada de dicionário de dados, atende ao requisito de transparência das aplicações previsto nos objetivos de aprendizagem da disciplina.

10 RISCOS E MEDIDAS DE CONTENÇÃO

- **Instabilidade do ambiente de laboratório.** Provisionar todo o ambiente por contêineres e manter um roteiro de instalação verificado por todas as equipes na primeira iteração.
- **Curva de aprendizado da camada cliente.** Antecipar a apresentação da tecnologia de interface e permitir que a equipe de aplicações inicie o trabalho com dados fictícios antes da conclusão da interface de programação.
- **Baixa adesão à prática de testes.** Dedicar uma aula inteira ao primeiro ciclo de teste e condicionar a aceitação das entregas à existência de testes anteriores ao código.

- **Indisponibilidade da escola parceira.** Definir o parceiro até a quarta semana e manter alternativa interna, como ação no próprio campus durante evento institucional.
- **Bloqueio recíproco entre equipes.** Congelar o esquema de mensagens e a especificação da interface ao final da primeira iteração, com alteração apenas por solicitação revisada.
- **Concentração do trabalho em poucos integrantes.** Distribuir tarefas com granularidade pequena, manter registro de contribuição no repositório e aplicar avaliação individual.
- **Atraso no cronograma.** Reduzir o escopo pela ordem inversa de prioridade, preservando obrigatoriamente broker, ingestão, persistência, interface de programação e aplicativo em pelo menos duas plataformas.

11 CRITÉRIOS DE ACEITAÇÃO DO PRODUTO

O produto é considerado aceito quando as seguintes condições forem simultaneamente satisfeitas:

- O aplicativo é executado em dispositivo móvel, navegador e computador pessoal a partir da mesma base de código.
- Os eventos gerados no aplicativo chegam ao painel em tempo real dentro do limite estabelecido.
- A base de dados armazena pelo menos um milhão de registros e responde às consultas dos painéis dentro do limite estabelecido.
- A suíte de testes é executada integralmente pelo pipeline de integração contínua sem falhas.
- O ensaio de carga é concluído sem perda de mensagens em qualidade de serviço 1.
- A documentação técnica, o dicionário de dados e o termo de consentimento estão publicados no repositório.
- A ação de divulgação é realizada e a devolutiva é entregue aos parceiros.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BECK, K. TDD: desenvolvimento guiado por testes. Porto Alegre: Bookman, 2010.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais. Brasília: Diário Oficial da União, 2018.

CHACON, S.; STRAUB, B. Pro Git. 2. ed. Nova York: Apress, 2014.

COULOURIS, G. et al. Sistemas distribuídos: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

KLEPPMANN, M. Designing data-intensive applications. Sebastopol: O'Reilly, 2017.

OASIS. MQTT version 3.1.1: OASIS standard. Burlington: OASIS, 2014.

PRESSMAN, R. S.; MAXIM, B. R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, I. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.

TANENBAUM, A. S.; VAN STEEN, M. Distributed systems. 4. ed. Amsterdã: Maarten van Steen, 2023.